

Execution Feedback Compression for Efficient Neural Program Repair

Deepak Naik M. V. and J. Swaminathan
Department of Computer Science and Engineering
Amrita School of Computing, Amrita Vishwa Vidyapeetham
Amritapuri, India

Abstract—Execution feedback from compilers, runtimes, and test cases provides valuable information for automatic program repair, but raw feedback is often verbose, noisy, and inconsistent across languages and execution environments. This work investigates whether compressed representations of execution feedback can preserve repair-relevant information while reducing context length and improving neural repair effectiveness. We compare raw feedback against categorical, localized, structured, and natural-language compressed variants under controlled repair settings.

Index Terms—automatic program repair, neural code intelligence, execution feedback, compiler diagnostics, program repair, feedback compression

I. Introduction

Automatic program repair aims to transform incorrect programs into correct ones. Recent neural repair systems increasingly use execution feedback, such as compiler errors, runtime exceptions, and failed test outcomes, to guide repair generation. However, raw execution feedback may contain redundant paths, environment-specific details, long stack traces, and messages that are not directly useful for repair.

This paper studies execution feedback compression as a supervision and prompting strategy for neural program repair. The central hypothesis is that compressed feedback can preserve the information needed for repair while reducing input length and noise.

II. Research Questions

RQ1: Information sufficiency. Which components of execution feedback are necessary and sufficient for neural program repair?

RQ2: Compression-efficiency tradeoff. How much can execution feedback be compressed before repair performance degrades?

RQ3: Error-type dependence. Do different bug categories require different feedback information, such as localization for runtime errors and counterexamples for wrong answers?

RQ4: Model dependence. Are feedback-compression effects consistent across model families and prompting or fine-tuning settings?

RQ5: Generalization. Do feedback-compression effects generalize beyond the primary Project CodeNet-derived dataset to external repair benchmarks?

TABLE I

Feedback representations compared in the study. All repair targets are evaluated under the same six feedback conditions.

Condition	Information provided to the repair model
None	Buggy code only.
Raw	Uncompressed status, diagnostic, or failure text.
Categorical	Normalized error type such as WA, CE, RE, TLE, or MLE.
Localized	Error type plus available line or diff-span context.
Structured	Machine-readable fields for error type, location, and normalized message.
Natural Language	Short human-readable explanation of the failure.

III. Feedback Representations

We compare the following feedback settings:

- **None:** The model receives only the buggy program.
- **Raw:** The model receives the full compiler, runtime, or test feedback.
- **Categorical:** The feedback is reduced to an error category and normalized error class.
- **Localized:** The feedback includes error type, line number, and nearby source-code context.
- **Structured:** Feedback is represented as machine-readable fields such as error type, line, symbol, and normalized message.
- **Natural Language:** Feedback is rewritten as a short human-readable explanation.

IV. Dataset Construction

The study uses a Project CodeNet-derived bug-fix dataset containing user error submissions paired with the user’s first accepted solution. Each record includes a problem identifier, user identifier, programming language, buggy submission, accepted solution, buggy execution status, code diffs, and test cases when available.

An initial inventory of the local dataset contains 2,223,631 bug-fix records, of which 2,156,595 include embedded test cases. The largest languages are C++ with 1,293,708 records, Python with 510,007 records, C with 130,164 records, and Java with 123,534 records. The main buggy status categories are Wrong Answer,

TABLE II
Large-scale corpus by problem-level split.

Split	Records	Problems
Train	80,031	1,991
Validation	9,517	249
Test	8,452	249
Total	98,000	2,489

Runtime Error, Time Limit Exceeded, Compile Error, and Presentation Error.

The large-scale corpus used in this study contains 98,000 records sampled from Python, Java, C++, and C, with exactly 24,500 records per language. Sampling is stratified by language and normalized online-judge failure category. We explicitly separate Wrong Answer (WA), Compile Error (CE), Runtime Error (RE), Time Limit Exceeded (TLE), and Memory Limit Exceeded (MLE). MLE is retained as a separate rare stratum rather than merged into an “other” category. Python has no compile-error stratum in the filtered corpus, so its language budget is assigned to WA, RE, TLE, and MLE. We use problem-level splits to prevent the same programming task from appearing in both training and evaluation. An automated split audit found zero overlapping problem identifiers across the train, validation, and test splits.

V. Methodology

For each buggy program, we execute the program against available tests and collect feedback. The feedback is then transformed into multiple representations. Repair generation is evaluated under identical conditions across feedback variants.

A. Pipeline

The implementation uses a streaming pipeline so that large JSONL datasets do not need to be loaded into memory. Records are selected using bounded reservoir sampling within language-status cells, then assigned to train, validation, and test splits by problem identifier. For the multilingual corpus, feedback is derived from online-judge status and diff metadata so that all three languages can be processed consistently before compiler toolchains are introduced. Execution-grade Python feedback and compiler-backed Java/C++ feedback are treated as stricter follow-up settings rather than prerequisites for corpus construction.

- 1) Stream bug-fix records from the dataset.
- 2) Select records matching the target languages, error categories, code-length limits, and test availability requirements.
- 3) Split records by problem identifier into train, validation, and test sets.
- 4) Convert status and diff metadata into raw, categorical, localized, structured, and natural-language feedback variants.

- 5) Build full-program repair targets and line-patch repair targets.
- 6) Store feedback artifacts, repair datasets, and split audits for downstream repair experiments.

B. Execution Feedback Collection

The multilingual feedback pipeline records online-judge status, language, diff-span metadata, and normalized feedback fields. When language toolchains are available, the stricter execution pipeline additionally records compile status, runtime status, test outcomes, error messages, stack traces, failing line information where available, and execution result categories.

C. Feedback Compression

Raw feedback is transformed using deterministic rules in the pilot phase. Later experiments may compare rule-based compression with model-generated compression.

D. Repair Generation

Repair generation uses two target formulations. In full-program repair, the model generates the complete accepted solution. In line-patch repair, the model generates a compact edit script that is applied to the buggy program. The current line-patch corpus contains 78,069 training instances, 9,299 validation instances, and 8,171 test instances after excluding empty patches and patches longer than the configured maximum. Training and evaluation scripts are resume-safe: interrupted model training resumes from the latest checkpoint, and generation or evaluation reruns append only missing record identifiers unless explicitly overwritten.

VI. Evaluation Metrics

We evaluate:

- compile success rate,
- execution pass rate,
- Pass@1,
- feedback token length,
- compression ratio,
- repair edit distance,
- performance by error category.

VII. Large-Scale Study Design

The final study is designed as a large-scale controlled experiment rather than a pilot report. Pilot experiments are used only to refine the pipeline, select feasible models, and identify threats to validity; pilot values are not reported as evidence in the paper.

A. Primary Dataset

The primary analysis uses the 98,000-record Project CodeNet-derived multilingual repair corpus described in Tables II–IV. Records are filtered to retain submissions with buggy code, accepted code, language metadata, online-judge status, code-diff metadata, and available tests. Dataset-level online-judge labels are retained as the

TABLE III
Large-scale corpus by language and error type.

Language	WA	CE	RE	TLE	MLE	Total
C	6,086	6,086	6,086	6,086	156	24,500
C++	5,193	5,193	5,192	5,193	3,729	24,500
Java	5,894	5,894	5,893	5,893	926	24,500
Python	8,058	0	8,058	8,059	325	24,500
Total	25,231	17,173	25,229	25,231	5,136	98,000

TABLE IV
Error-type distribution across problem-level splits.

Split	WA	CE	RE	TLE	MLE	Total
Train	21,011	14,182	20,316	20,359	4,163	80,031
Validation	2,142	1,588	2,689	2,726	372	9,517
Test	2,078	1,403	2,224	2,146	601	8,452

TABLE V
Feedback length statistics on the validation split.

Condition	Mean tokens	Median tokens	Raw/condition
None	0.0	0.0	—
Raw	12.9	12.0	1.00x
Categorical	2.3	2.0	5.53x
Localized	24.8	24.0	0.52x
Structured	45.7	45.0	0.28x
Natural language	17.3	17.0	0.74x

common multilingual feedback source. Locally observed execution behavior is used in stricter language-specific analyses where the required runtime or compiler toolchain is available.

B. Experimental Conditions

For each repair instance, the same buggy program and evaluation tests will be paired with multiple feedback representations: no feedback, raw feedback, categorical feedback, localized feedback, structured feedback, and natural-language compressed feedback. All feedback conditions will be evaluated on identical train/test splits and identical model settings. The study will include both full-program repair and edit-oriented repair targets where feasible.

C. Models and Baselines

The large-scale analysis will include at least three baseline families:

- No-feedback repair: the model receives only the buggy program.
- Raw-feedback repair: the model receives uncompressed execution feedback.
- Compressed-feedback repair: the model receives categorical, localized, structured, or natural-language compressed feedback.

Model families will include a compact fine-tuned code model and at least one instruction-tuned code model. The no-feedback and raw-feedback conditions are necessary

baselines: compressed feedback is only useful if it improves over no feedback or preserves raw-feedback performance at substantially lower context cost. Validation results are used for model and feedback-condition selection; the held-out test split is reserved for final reporting.

D. Statistical Analysis

Because every feedback condition is evaluated on the same repair instances, the main comparisons are paired. We will report Pass@1 with confidence intervals, exact McNemar tests for paired correctness differences, and effect sizes based on discordant pairs. Results will be stratified by locally observed failure type, code length, feedback length, and repair edit size.

E. External Validation

To test generalization beyond the primary corpus, the study will include an external benchmark such as QuixBugs. External validation will be treated separately from the primary CodeNet-derived analysis because the benchmark distribution, tests, and repair style differ from competitive-programming submission histories.

VIII. Expected Result Tables

The final manuscript will report only large-scale results. Tables will include: dataset statistics after filtering, feedback compression statistics, repair performance by feedback representation and model family, paired statistical comparisons, error-type stratification, and external benchmark performance.

TABLE VI
Planned repair-effectiveness breakdown. Each cell will report Pass@1 with confidence intervals on the held-out test split.

Target	Feedback	WA	CE	RE	TLE	MLE	Overall
Full program	None	–	–	–	–	–	–
Full program	Raw	–	–	–	–	–	–
Full program	Categorical	–	–	–	–	–	–
Full program	Localized	–	–	–	–	–	–
Full program	Structured	–	–	–	–	–	–
Full program	Natural language	–	–	–	–	–	–
Line patch	None	–	–	–	–	–	–
Line patch	Raw	–	–	–	–	–	–
Line patch	Categorical	–	–	–	–	–	–
Line patch	Localized	–	–	–	–	–	–
Line patch	Structured	–	–	–	–	–	–
Line patch	Natural language	–	–	–	–	–	–

IX. Threats to Validity

The study may be affected by dataset bias, language-specific diagnostic formats, benchmark-specific test coverage, and model sensitivity to prompt wording. These threats will be expanded after the pilot experiments.

X. Conclusion

This work explores whether execution feedback can be compressed into compact, repair-relevant representations for neural program repair. If successful, the approach can reduce context cost, improve robustness, and provide a practical bridge between execution-grounded program analysis and neural repair models.

References

- [1] T. B. Updated, “Placeholder reference,” To Be Updated, 2026.